

Program No:01**Class: II BCA**

Purpose: Write a program create list with N elements. Find all unique elements in the list. If an element is found only once in the list, then add that element to the unique list.

```
num=[]
ulist=[]
n=int(input("Enter number of elements to be insert:\n"))
for i in range(n):
    ele=input("Enter the element")
    num.append(ele)
print("the elements in the list are ")
print(num)
for e in num:
    if(num.count(e)==1):
        ulist.append(e)
print("The Unique elements in the list are:\n",ulist)
```

OUTPUT:

Enter number of elements to be insert:

5

Enter the element 4

Enter the element 5

Enter the element 3

Enter the element 5

Enter the element 2

the elements in the list are

[' 4', ' 5', ' 3', ' 5', ' 2']

The Unique elements in the list are:

[' 4', ' 3', ' 2']

Program No:02**Class: II BCA****Purpose: Program using user-defined functions to find the area of rectangle, square, circle, and triangle by accepting suitable input parameters from user.**

```
def arect():
    l=int(input("Enter the length of rectangle : "))
    w=int(input("Enter the width of rectangle : "))
    a1=l*w
    print("The area of rectangle is: ",a1)
def asquare():
    s=int(input("Enter the side: "))
    a2=s*s
    print("The area of square is: ",a2)
def acir():
    r=int(input("Enter the radius of the circle: "))
    a3=3.14*(r*r)
    print("Area of circle :",a3)
def atri():
    b=int(input("Enter the base of triangle : "))
    h=int(input("Enter the height of triangle : "))
    a4=1/2*(b*h)
    print("The area of square is: ",a4)
while(True):
    print("Choose your option ")
    print("1. area of Rectangle")
    print("2. area of Square")
    print("3. area of Circle")
    print("4. area of Triangle")
    print("5. To Exit")
    i=int(input("Enter the choice : "))
```

```
if i==1:
    arect()
elif i==2:
    asquare()
elif i==3:
    acir()
elif i==4:
    atri()
elif i==5:
    break
else:
    print("Invalid option")
```

OUTPUT:

Choose your option

1. area of Rectangle
2. area of Square
3. area of Circle
4. area of Triangle
5. To Exit

Enter the choice : 1

Enter the length of rectangle : 3

Enter the width of rectangle : 4

The area of rectangle is: 12

Choose your option

1. area of Rectangle
2. area of Square
3. area of Circle
4. area of Triangle
5. To Exit

Enter the choice : 2

Enter the side: 4

The area of square is: 16

Choose your option

1. area of Rectangle
2. area of Square
3. area of Circle
4. area of Triangle
5. To Exit

Enter the choice : 3

Enter the radius of the circle: 4

Area of circle : 50.24

Choose your option

1. area of Rectangle
2. area of Square
3. area of Circle
4. area of Triangle
5. To Exit

Enter the choice : 4

Enter the base of triangle : 4

Enter the height of triangle : 2

The area of square is: 4.0

Choose your option

1. area of Rectangle
2. area of Square
3. area of Circle
4. area of Triangle
5. To Exit

Enter the choice : 5

Program No:03**Class: II BCA****Purpose: Consider a tuple t1= (1,2,5,7,9,2,4,6,8,10). Write a program to perform following operations:**

- a) Print half the values of tuple in one line and the other half in the next line.**
- b) Print another tuple whose values are even numbers in the given tuple.**
- c) Concatenate a tuple t2= (11, 13, 15) with t1.**
- d) Return maximum and minimum value from this tuple.**

```
t1=(1,2,5,7,9,2,4,6,8,10)
t2=(11,13,15)
tp1=t1[:5]
tp2=t1[5:]
print(tp1)
print(tp2)
print("the even numbers in this tuples are:")
for i in range(len(t1)):
    if(t1[i]%2==0):
        print(t1[i],end=" ")
res=t1+t2
print("\n the tuple after concation are:\n",res)
print("maximaum value=",max(res))
print("minimum value=",min(res))
```

OUTPUT:

```
(1, 2, 5, 7, 9)
(2, 4, 6, 8, 10)
the even numbers in this tuples are:
2 2 4 6 8 10
the tuple after concation are:
(11, 13, 15)
maximaum value= 15
minimum value= 11
```

Program No:04**Class: II BCA**

Purpose: Write a function that takes a sentence as input from the user and calculates the frequency of each letter. Use a variable of dictionary type to maintain the count.

```
def CountFrequency(my_list):  
    freq={}  
    for item in my_list:  
        if(item in freq):  
            freq[item]+=1  
        else:  
            freq[item]=1  
  
    for key,value in freq.items():  
        print("%s.%d"%(key,value))  
my_list=input("Enter the sentence:")  
CountFrequency(my_list)
```

OUTPUT:

```
Enter the sentence:Sacred hEart CoLleGE  
S.1  
a.2  
c.1  
r.2  
e.2  
d.1  
.3  
h.1  
E.2  
t.1  
C.1  
o.1  
L.1  
l.1
```

Program No:05**Class: II BCA**

Purpose: Write a function nearly equal to test whether two strings are nearly equal. Two strings a and b are nearly equal if one character change in b results in string a.

```
s1=input("Enter First string : ")
s2=input("Enter second string : ")
if s1 == s2:
    print("Two Strings are Equal")
if len(s1) != len(s2):
    print("Two Strings are Not Equal")
    exit()
else:
    count=[]
    for i in range(len(s1)):
        if s1[i]!=s2[i]:
            count.append(i)
    if (len(count)>1):
        print("Two strings are not equal")
    else:
        print("Two strings are nearly equal")
        for i in count:
            print(f'At Index {i}-->'{s1[i]}' changed to '{s2[i]}''")
```

OUTPUT:

```
Enter First string : cat
Enter second string : bat
Two strings are nearly equal
At Index 0-->'c' changed to 'b'
```

```
runfile('D:/Backup/pa5.py', wdir='D:/Backup')
Enter First string : kavya
Enter second string : navya
Two strings are nearly equal
At Index 0-->'k' changed to 'n'
```

```
runfile('D:/Backup/pa5.py', wdir='D:/Backup')
Enter First string : cat
Enter second string : cat
Two Strings are Equal
Two strings are nearly equal
```

```
Enter First string : manaswi
Enter second string : mamatha
Two strings are not equal
```

Program No:06**Class: II BCA****Purpose: Write a program to create a text file and compute the number of characters, words and lines in a file.**

```
f=open('file.txt','w')
print('content of file is ')
f.write('sacred heart college madanthyar')
f=open('file.txt','r')
c=f.read()
print(c)
f=open('file.txt','r')
c=f.read().replace("\n"," ")
word=0
lines=0
ch=0
f=open("file.txt",'r')
for i in f:
    word+=len(i.split())
    lines+=1
ch=len(c)-c.count(" ")
print("no of words:",word)
print("no of lines:",lines)
print("no of charecters:",ch)
```

OUTPUT:

```
content of file is
sacred heart college madanthyar
no of words: 4
no of lines: 1
no of charecters: 28
```

Program No:07**Class: II BCA**

Purpose: Write a Pandas program to join the two given data frames along rows.
Sample Data frame may contain details of student like roll no, name, Total Marks.

```
import pandas as pd
# Sample data frames
data1 = {'Rollno': [101, 102, 103],
        'Student_Name': ['Akhil', 'Basu',
        'Charlie'],
        'Total Marks': [450, 480, 430]}
df1 = pd.DataFrame(data1)
pd.DataFrame(data1, index=[0, 1, 2])
data2 = {'Rollno': [104, 105, 106],
        'Student_Name': ['Dhoni', 'Eva', 'Faruq'],
        'Total Marks': [410, 470, 440]}

df2 = pd.DataFrame(data2)
print(df1, "\n\n", df2)
result_df = pd.concat([df1, df2], ignore_index=True)
pd.concat([df1, df2])
print("\n\n", "Joined Data Frame:\n3")
print(result_df)
```

OUTPUT:

	Rollno	Student_Name	Total Marks
0	101	Akhil	450
1	102	Basu	480
2	103	Charlie	430

	Rollno	Student_Name	Total Marks
0	104	Dhoni	410
1	105	Eva	470
2	106	Faruq	440

Joined Data Frame:
3

	Rollno	Student_Name	Total Marks
0	101	Akhil	450
1	102	Basu	480
2	103	Charlie	430
3	104	Dhoni	410
4	105	Eva	470
5	106	Faruq	440

Program No:01

Class: II BCA

Purpose: Program to create a class Employee with empno, name, depname, designation, age and salary and perform the following function.

- i) Accept details of N employees**
- ii) Search given employee using empno**
- iii) Display employee details in neat format.**

```
class emp:
    def __init__(self):
        self.empno=None
        self.empname=None
        self.desg=None
        self.depname=None
        self.age=None
        self.sal=None

    def getemp(self):
        self.empno=(int(input("Enter id : ")))
        self.empname=(input("Enter name : "))
        self.desg=(input("Enter designation:"))
        self.depname=(input("Enter department: "))
        self.age= (int(input("Enter age: ")))
        self.sal=(int(input("Enter salary: ")))

    def dispemp(self):
        print("-"*30)
        print(f'Employ Number: {self.empno}')
        print(f'Employ Name: {self.empname}')
        print(f'Employ age: {self.age}')
        print(f'Employ department: {self.depname}')
        print(f'Employ designation: {self.desg}')
        print(f'Employ salary: {self.sal}')
        print("-"*15)

    def search(self, empno):
        if empno == self.empno:
            return 1
        else:
            return 0

n = int(input("Enter Total Number of Employees:"))
L = []

for i in range(n):
    E = emp()
```

```

E.getemp()
L.append(E)

for item in L:
    item.dispemp()

empid = int(input("Enter Employ Id to Search:"))
found = 0
for e1 in L:
    found = e1.search(empid)
    if (found==1):
        print("Employee found")
        e1.dispemp()
        break
if (found==0):
    print("Employee not found")

```

OUTPUT:

Enter Total Number of Employees:2

Enter id : 101

Enter name : kavya

Enter designation:IT

Enter department: Manager

Enter age: 29

Enter salary: 50000

Enter id : 102

Enter name : manasa

Enter designation:HR

Enter department: Business

Enter age: 30

Enter salary: 40000

Employ Number:101

Employ Name:kavya

Employ age:29

Employ department:Manager

Employ designation:IT

Employ salary:50000

Employ Number:102

Employ Name:manasa

Employ age:30

Employ department:Business

Employ designation:HR

Employ salary:40000

Enter Employ Id to Search:102

Employee found

Employ Number:102

Employ Name:manasa

Employ age:30

Employ department:Business

Employ designation:HR

Employ salary:40000

Enter Total Number of Employees:2

Enter id : 101

Enter name : sakshi

Enter designation:IT

Enter department: manager

Enter age: 30

Enter salary: 40000

Enter id : 102

Enter name : mamatha

Enter designation:Sales

Enter department: manager

Enter age: 40

Enter salary: 30000

Employ Number:101

Employ Name:sakshi

Employ age:30

Employ department:manager

Employ designation:IT

Employ salary:40000

Employ Number:102

Employ Name:mamatha

Employ age:40

Employ department:manager

Employ designation:Sales

Employ salary:30000

Enter Employ Id to Search:103

Employee not found

Program No:02

Class: II BCA

Purpose: Write a program menu driven to create a Bank Account class. class should support the following methods for

- i) Deposit**
- ii) Withdraw**
- iii) Get Balance.**

Create a subclass Savings Account class that behaves just like a Bank Account, but also has an interest rate and a method that increases the balance by the appropriate amount of interest.

```
class BankAccount:
    def __init__(self,name,accno, balance):
        self.name=name
        self.accno=accno
        self.balance = balance
    def deposit(self, amount):
        self.balance += amount
    def withdraw(self, amount):
        if self.balance < amount:
            print("Insufficient balance")
        else:
            self.balance -= amount
            print("Withdrawal successful")
    def get_balance(self):
        return self.balance
    def cust_disp(self):
        print(f'Customer Name: {self.name}')
        print(f'Account Number: {self.accno}')
        print(f'Balance: {self.balance}')

class SavingsAccount(BankAccount):
    def add_interest(self):
        self.r=float(input("Enter interest rate: "))
        interest = self.balance * self.r/100
        self.balance += interest
        print(f'After adding intrest to your account,Balance is: {self.balance}')

n=input("Enter the Customer Name:")
a=int(input("Enter the Account Number"))
b=int(input("Enter the balance"))
account =SavingsAccount(n,a,b)

while True:
    print("\nBank Account Menu")
    print("1. Deposit")
```

```

print("2. Withdraw")
print("3. Get Balance")
print("4. Add Interest")
print("5. Display")
print("6. Quit")
choice = int(input("Enter your choice (1-6): "))
if choice == 1:
    amount1 = float(input("Enter amount to deposit: "))
    account.deposit(amount1)

elif choice == 2:
    amount = float(input("Enter amount to withdraw: "))
    account.withdraw(amount)
elif choice == 3:
    print("Balance: ", account.get_balance())
elif choice == 4:
    account.add_interest()
elif choice == 5:
    account.cust_disp()
elif choice == 6:
    break
else:
    print("Invalid choice")

```

OUTPUT:

Enter the Customer Name:deepthi
 Enter the Account Number:324156
 Enter the balance:5000

Bank Account Menu

1. Deposit
 2. Withdraw
 3. Get Balance
 4. Add Interest
 5. Display
 6. Quit
 Enter your choice (1-6): 1
 Enter amount to deposit: 500

Bank Account Menu

1. Deposit
 2. Withdraw
 3. Get Balance
 4. Add Interest
 5. Display
 6. Quit
 Enter your choice (1-6): 2

Enter amount to withdraw: 500

Withdrawal successful

Bank Account Menu

1. Deposit
2. Withdraw
3. Get Balance
4. Add Interest
5. Display
6. Quit

Enter your choice (1-6): 3

Balance: 5000.0

Bank Account Menu

1. Deposit
2. Withdraw
3. Get Balance
4. Add Interest
5. Display
6. Quit

Enter your choice (1-6): 4

Enter interest rate: 3

After adding intrest to your account,Balance is:5150.0

Bank Account Menu

1. Deposit
2. Withdraw
3. Get Balance
4. Add Interest
5. Display
6. Quit

Enter your choice (1-6): 5

Customer Name:deepthi

Account Number:324156

Balance: 5150.0

Bank Account Menu

1. Deposit
2. Withdraw
3. Get Balance
4. Add Interest
5. Display
6. Quit

Enter your choice (1-6): 6

Program No:03

Class: II BCA

Purpose: Create a GUI to input Principal amount, rate of interest and number of years, Calculate Compound interest. When button submit is pressed Compound interest should be displayed in a textbox. When clear button is pressed all contents should be cleared.

```
from tkinter import*
def clear_all():
    principle_field.delete(0,END)
    rate_field.delete(0,END)
    time_field.delete(0,END)
    compound_field.delete(0,END)
    principle_field.focus_set()
def calculate_ci():
    principle=int(principle_field.get())
    rate=float(rate_field.get())
    time=int(time_field.get())
    CI=principle*pow((1+rate/100),time)
    compound_field.insert(10,CI)
if __name__ == "__main__":
    root=Tk()
    root.configure(background="light green")
    root.geometry("400x250")
    root.title("Compound intrest calculator")
    label1=Label(root,text="Principle Amount(Rs):",fg='black',bg='red')
    label2=Label(root,text="Rate(%):",fg='black',bg='red')
    label3=Label(root,text="Time(years):",fg='black',bg='red')
    label4=Label(root,text="Compound intrest",fg='black',bg='red')

    label1.grid(row=1,column=0,padx=10,pady=10)
    label2.grid(row=2,column=0,padx=10,pady=10)
    label3.grid(row=3,column=0,padx=10,pady=10)
    label4.grid(row=5,column=0,padx=10,pady=10)

    principle_field=Entry(root)
    rate_field=Entry(root)
    time_field=Entry(root)
    compound_field=Entry(root)

    principle_field.grid(row=1,column=1,padx=10,pady=10)
    rate_field.grid(row=2,column=1,padx=10,pady=10)
    time_field.grid(row=3,column=1,padx=10,pady=10)
    compound_field.grid(row=5,column=1,padx=10,pady=10)

    button1=Button(root,text="Submit",bg="red",fg="black",command=calculate_ci)
```

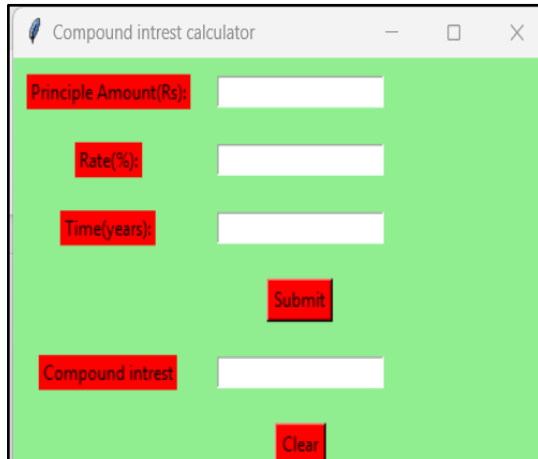
```
button2=Button(root,text="Clear",bg="red",fg="black",command=clear_all)
```

```
button1.grid(row=4,column=1,pady=10)
```

```
button2.grid(row=6,column=1,pady=10)
```

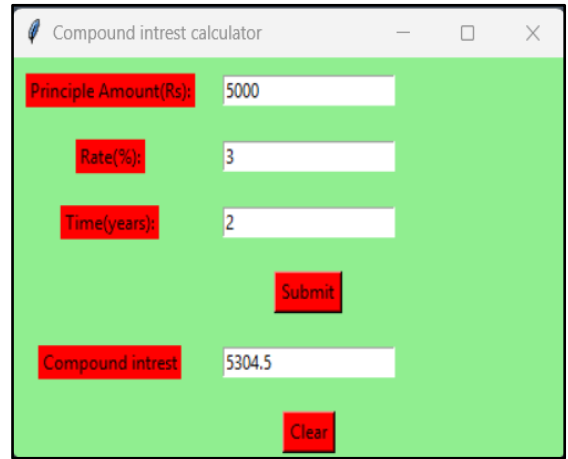
```
root.mainloop()
```

OUTPUT:



The screenshot shows a window titled "Compound interest calculator" with a light green background. It contains the following elements:

- Principle Amount(Rs):
- Rate(%):
- Time(years):
- Submit:
- Compound interest:
- Clear:



The screenshot shows the same window after calculation. The input fields are filled with the values 5000, 3, and 2. The Compound interest field displays the result 5304.5.

- Principle Amount(Rs):
- Rate(%):
- Time(years):
- Submit:
- Compound interest:
- Clear:

Program No:04**Class: II BCA****Purpose: Write a GUI program to implement simple calculator.**

```
from tkinter import*
expression=" "
#Function to update expression in the text entry box
def press(num):
    global expression
    expression=expression+str(num)
    equation.set(expression)
#Function to evaluate the final expression
def equalPress():
    try:
        global expression
        total=str(eval(expression))
        equation.set(total)
        expression=" "
    except:
        equation.set("Error")
        expression=" "
#Function to clear the contents of textentry box
def clear():
    global expression
    expression=" "
    equation.set(" ")
#Driver Code
if __name__=="__main__":
    #create a GUI window
    gui=Tk()
    gui.configure(background="light green")
    gui.title("Simple Calculator")
    gui.geometry("270x150")
    equation=StringVar()
    expression_field=Entry(gui,textvariable=equation)
    expression_field.grid(columnspan=4,ipadx=70)

    button1=Button(gui,text="1",fg="black",bg="red",command=lambda:press(1),height=1,width=7)
    button1.grid(row=2,column=0)

    button2=Button(gui,text="2",fg="black",bg="red",command=lambda:press(2),height=1,width=7)
    button2.grid(row=2,column=1)
```

```

button3=Button(gui,text="3",fg="black",bg="red",command=lambda:press(3),height=1,width=7)
    button3.grid(row=2,column=2)

button4=Button(gui,text="4",fg="black",bg="red",command=lambda:press(4),height=1,width=7)
    button4.grid(row=3,column=0)

button5=Button(gui,text="5",fg="black",bg="red",command=lambda:press(5),height=1,width=7)
    button5.grid(row=3,column=1)

button6=Button(gui,text="6",fg="black",bg="red",command=lambda:press(6),height=1,width=7)
    button6.grid(row=3,column=2)

button7=Button(gui,text="7",fg="black",bg="red",command=lambda:press(7),height=1,width=7)
    button7.grid(row=4,column=0)

button8=Button(gui,text="8",fg="black",bg="red",command=lambda:press(8),height=1,width=7)
    button8.grid(row=4,column=1)

button9=Button(gui,text="9",fg="black",bg="red",command=lambda:press(9),height=1,width=7)
    button9.grid(row=4,column=2)

button0=Button(gui,text="0",fg="black",bg="red",command=lambda:press(0),height=1,width=7)
    button0.grid(row=5,column=0)

plus=Button(gui,text="+",fg="black",bg="red",command=lambda:press("+"),height=1,width=7)
    plus.grid(row=2,column=3)
    minus=Button(gui,text="-",fg="black",bg="red",command=lambda:press("-"),height=1,width=7)
    minus.grid(row=3,column=3)

multiply=Button(gui,text="*",fg="black",bg="red",command=lambda:press("*"),height=1,width=7)
    multiply.grid(row=4,column=3)

divide=Button(gui,text="/",fg="black",bg="red",command=lambda:press("/"),height=1,width=7)
    divide.grid(row=5,column=3)

equal=Button(gui,text="=",fg="black",bg="red",command=equalPress,height=1,width=7)

```

```
equal.grid(row=5,column=2)
```

```
clear=Button(gui,text="clear",fg="black",bg="red",command=clear,height=1,width=7)
```

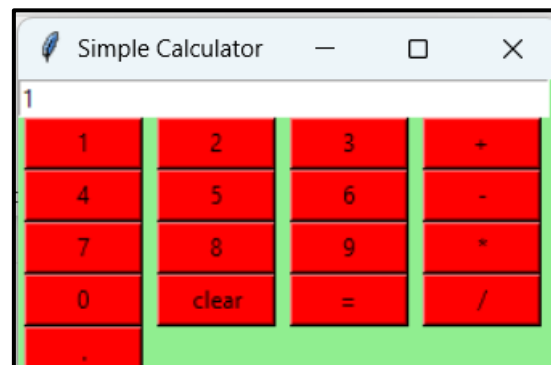
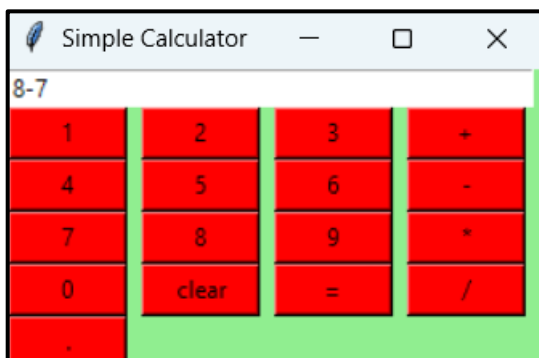
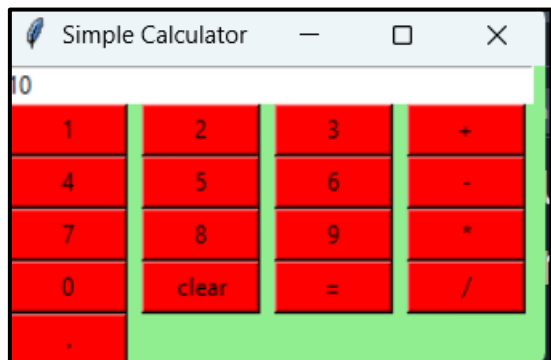
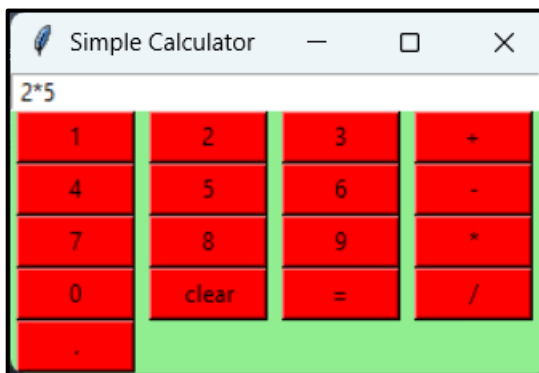
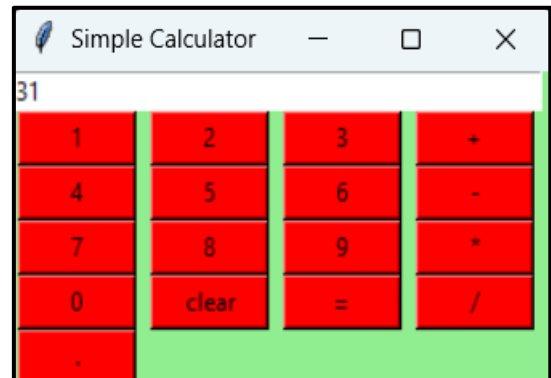
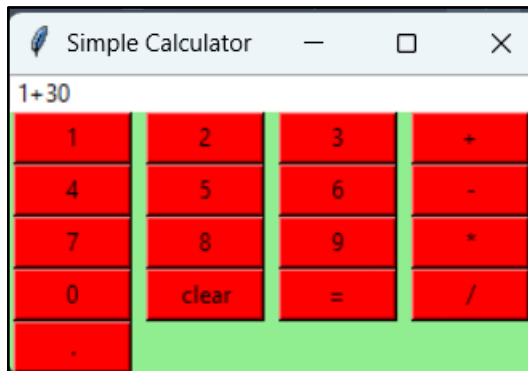
```
clear.grid(row=5,column=1)
```

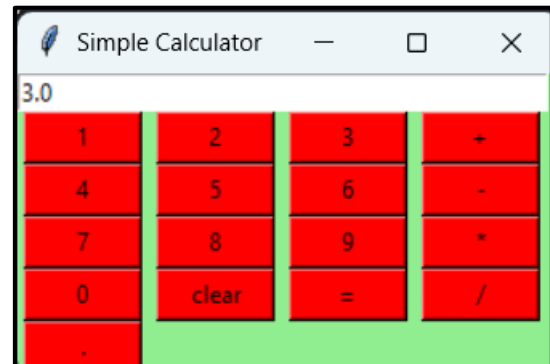
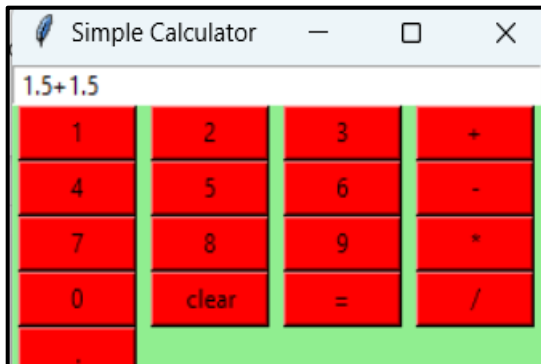
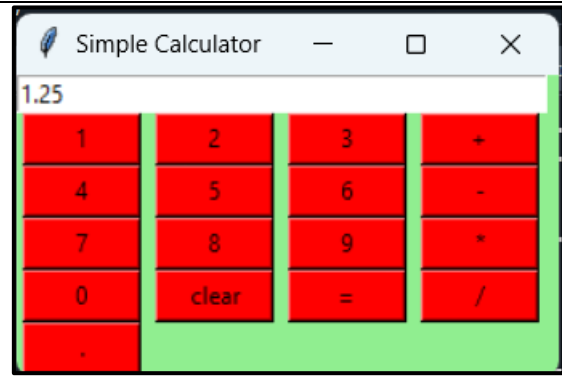
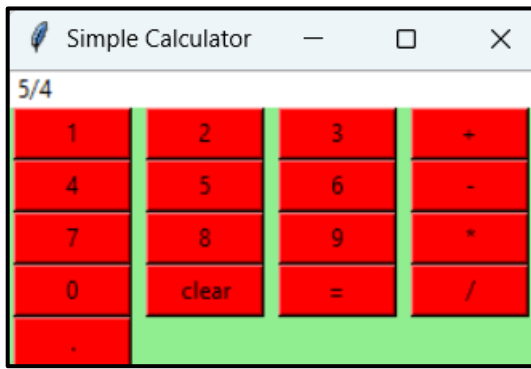
```
Decimal=Button(gui,text=".",fg="black",bg="red",command=lambda:press("."),height=1,  
width=7)
```

```
Decimal.grid(row=6,column=0)
```

```
gui.mainloop()
```

OUTPUT:





Program No:05

Class: II BCA

Purpose: Create a table student table (regno, name and marks in 3 subjects) using MySQL/SQLite and perform the followings.

- a. To accept the details of students and store it in database.**
- b. To display the details of all the students**
- c. Delete particular student record using regno.**

```
import sqlite3
con = sqlite3.connect('college.db')
c1=con.cursor()
c1.execute("""create table if not exists student
            (regno int primary key,
             name text not null,
             sub1 int not null,
             sub2 int not null,
             sub3 int not null)""")
print("Student Table created successfully")
def insert_student():
    try:
        regno = int(input("Enter registration number: "))
        name = input("Enter name: ")
        sub1 = int(input("Enter marks for subject 1: "))
        sub2 = int(input("Enter marks for subject 2: "))
        sub3 = int(input("Enter marks for subject 3: "))
        if (sub1>100 or sub2>100 or sub3 > 100):
            raise ValueError
        c1.execute("""insert into student values(?,?,?,?)""",(regno,name,sub1,sub2,sub3))
        print("Student record created successfully")
        con.commit()
    except sqlite3.Error:
        print("please put correct unique regno data")
    except ValueError:
        print("please enter correct valid data")

def display_students():
    c1.execute("""select * from student""")
    record=c1.fetchall()
    if record==[]:
        print("No Records Found")
    else:
        print("(RegNo: Name: Sub1: Sub2: Sub3:)")
        for row in record:
            print(row)
```

```

def delete_student():
    r = int(input("Enter registration number to delete: "))
    c1.execute(f"select * from student where regno={r}")
    row = c1.fetchone()
    if row is None:
        print("No student found with that registration number")
    else:
        c1.execute(f"delete from student where regno={r}")
        con.commit()
        print("Student deleted successfully")
while True:
    print("1.Insert student record")
    print("2.Display all student records")
    print("3.Delete a student record")
    print("4.Exit")
    ch = int(input("Enter choice:"))
    if ch == 1:
        insert_student()
    elif ch == 2:
        display_students()
    elif ch == 3:
        delete_student()
    elif ch == 4:
        break
    else:
        print("Invalid choice")
con.commit()
con.close()

```

```

1.Insert student record
2.Display all student records
3.Delete a student record
4.Exit
Enter choice:1
Enter registration number: 173165022
Enter name: manass
Enter marks for subject 1: 80
Enter marks for subject 2: 90
Enter marks for subject 3: 80
Student record created successfully
1.Insert student record
2.Display all student records
3.Delete a student record
4.Exit
Enter choice:1
Enter registration number: 2345167
Enter name: kavya
Enter marks for subject 1: 90
Enter marks for subject 2: 70

```

Enter marks for subject 3: 60

Student record created successfully

1.Insert student record

2.Display all student records

3.Delete a student record

4.Exit

Enter choice:2

(RegNo: Name: Sub1: Sub2: Sub3:)

(173165022, 'manass', 80, 90, 80)

(2345167, 'kavya', 90, 70, 60)

1.Insert student record

2.Display all student records

3.Delete a student record

4.Exit

Enter choice:3

Enter registration number to delete: 2345167

Student deleted successfully

1.Insert student record

2.Display all student records

3.Delete a student record

4.Exit

Enter choice:2

(RegNo: Name: Sub1: Sub2: Sub3:)

(173165022, 'manass', 80, 90, 80)

1.Insert student record

2.Display all student records

3.Delete a student record

4.Exit

Enter choice:3

Enter registration number to delete: 173165022

Student deleted successfully

1.Insert student record

2.Display all student records

3.Delete a student record

4.Exit

Enter choice:2

No Records Found

1.Insert student record

2.Display all student records

3.Delete a student record

4.Exit

Enter choice:4

Program No:06

Class: II BCA

Purpose: Create a table employee (empno, name and salary) using MySQL/SQLite and perform the followings

- a. To accept the details of employees and store it in database.**
- b. To display the details of a specific employee**
- c. To display employee details whose salary lies within a certain range**

```
import sqlite3
con = sqlite3.connect('employee.db')
c1=con.cursor()
c1.execute("""create table if not exists emp
            (empno int primary key,
            name text not null,
            sal int not null)""")
print("Employee Table created successfully")
def insert_emp():
    try:
        e = int(input("Enter the employee number:"))
        n = input("Enter the name:")
        s = int(input("Enter the salary:"))
        c1.execute("""insert into emp values(?,?,?)""",(e,n,s))
        print("Employee record created successfully")
        con.commit()
    except sqlite3.Error:
        print("please put correct unique employee number")
    except ValueError:
        print("please enter correct valid data")

def display_all():
    c1.execute("""select * from emp""")
    record=c1.fetchall()
    if record==[]:
        print("No Records Found")
    else:
        print("(Employee No.: Name: Salary:)")
        for row in record:
            print(row)

def display_emp():
    eno=int(input("Enter the employee number to display"))
    c1.execute(f"select * from emp where empno={eno} ")
    record=c1.fetchone()
    if record is None:
        print("No employee found with employee No.")
```

```

else:
    print("(Employee No.: Name: Salary:)")
    #for row in record:
    print(record)

def sal_emp():
    r1 = int(input("Enter the starting range of salary"))
    r2 = int(input("Enter the ending range of salary"))
    c1.execute(f"select * from emp where sal between {r1} and {r2}")
    record=c1.fetchall()
    if record==[]:
        print("No Records Found")
    else:
        print("(Employee No.: Name: Salary:)")
        for row in record:

            print(row)

while True:
    print("1.Insert employee record")
    print("2.Display all employee records")
    print("3.Display specific employee details")
    print("4.Display employee details with salary range")
    print("5.Exit")
    ch=int(input("Enter choice:"))
    if ch==1:
        insert_emp()
    elif ch==2:
        display_all()
    elif ch==3:
        display_emp()
    elif ch==4:
        sal_emp()
    elif ch==5:
        break
    else:
        print("Invalid choice")
con.commit()
con.close()

```

OUTPUT:

Employee Table created successfully

- 1.Insert employee record
- 2.Display all employee records
- 3.Display specific employee details
- 4.Display employee details with salary range
- 5.Exit

Enter choice:1

Enter the employee number:12345

Enter the name:shrays

Enter the salary:40000

Employee record created successfully

- 1.Insert employee record
- 2.Display all employee records
- 3.Display specific employee details
- 4.Display employee details with salary range
- 5.Exit

Enter choice:1

Enter the employee number:34562

Enter the name:deepa

Enter the salary:50000

Employee record created successfully

- 1.Insert employee record
- 2.Display all employee records
- 3.Display specific employee details
- 4.Display employee details with salary range
- 5.Exit

Enter choice:2

(Employee No.: Name: Salary:)

(12345, 'shrays', 40000)

(34562, 'deepa', 50000)

- 1.Insert employee record
- 2.Display all employee records
- 3.Display specific employee details
- 4.Display employee details with salary range
- 5.Exit

Enter choice:3

Enter the employee number to display 12345

(Employee No.: Name: Salary:)

(12345, 'shrays', 40000)

- 1.Insert employee record
- 2.Display all employee records
- 3.Display specific employee details
- 4.Display employee details with salary range
- 5.Exit

Enter choice:4

Enter the starting range of salary 30000

Enter the ending range of salary 40000

(Employee No.: Name: Salary:)

(12345, 'shrays', 40000)

1.Insert employee record

2.Display all employee records

3.Display specific employee details

4.Display employee details with salary range

5.Exit

Enter choice:5

Program No:07

Class: II BCA

Purpose: Consider following data and draw the bar graph using matplotlib library. (Use CSV or Excel).

Batsman	2017	2018	2019	2020
Virat Kohli	2501	1855	2203	1223
Steve Smith	2340	2250	2003	1153
Babar Azam	1750	2147	1896	1008
Rohit Sharma	1463	1985	1854	1638
Kane Williamson	1256	1785	1874	1974
Jos Butler	1125	1853	1769	1436

Display appropriate title for axis and chart. Also show legends.

```
from tkinter import *
import matplotlib.pyplot as plt
import pandas as pd
def equalbut():
    d=pd.read_csv("D:/Backup/Bats.csv")
    d.plot.bar(x='Batsman')
    plt.title("Cricketers Score")
    plt.xlabel("Cricketer Name")
    plt.ylabel("Scores")
    plt.legend()
    plt.show()
def end():

    me.destroy()
me=Tk()
me.configure(background='light green')
me.geometry("300x300")
me.title("Bar Graph Visualization")
bt=Button(me,padx=14,pady=14,bd=4,bg='white',command=equalbut,text="Load the
data",font=("times",12,"bold"))
bt.place(x=10,y=80)
bt1=Button(me,padx=14,pady=14,bd=4,bg='white',command=end,text="Exit",font=("time
s",12,"bold"))
bt1.place(x=150,y=80)
me.mainloop()
```

OUTPUT:

